

LAB SESSION 6

Lab Name: System Design & UML Modeling

Project: MediScript-E — Digital Healthcare Platform

Objectives

- Design the system architecture of MediScript-E
- Apply UML modeling for software design documentation
- Translate requirements into a concrete system design

Theory

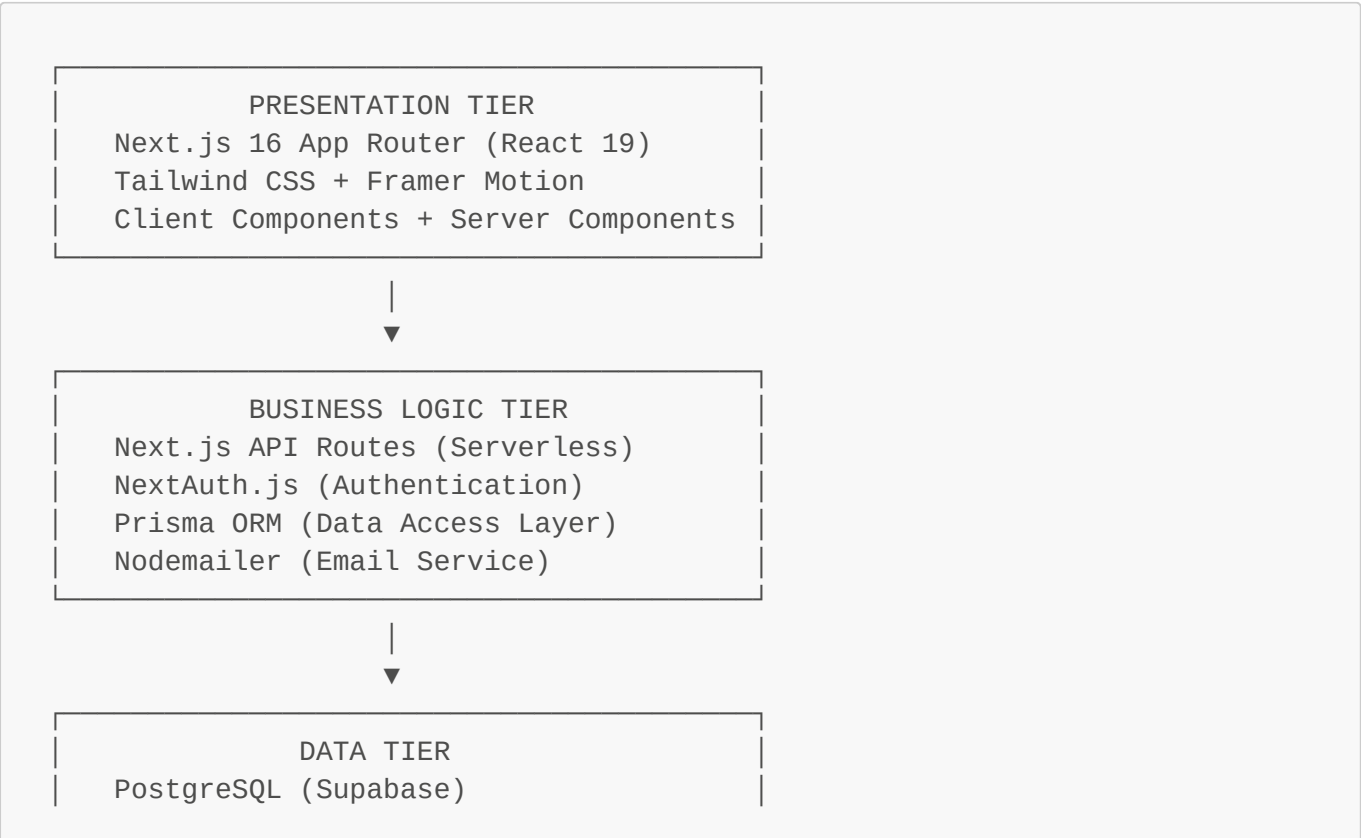
Software design defines how the system will be built. Good design ensures:

- **Maintainability:** Easy to modify and extend
- **Scalability:** Can handle growing users and data
- **Low Coupling:** Components are independent and reusable

UML (Unified Modeling Language) diagrams communicate design clearly to all stakeholders.

Task 1: System Architecture

MediScript-E follows a **3-Tier Architecture**:

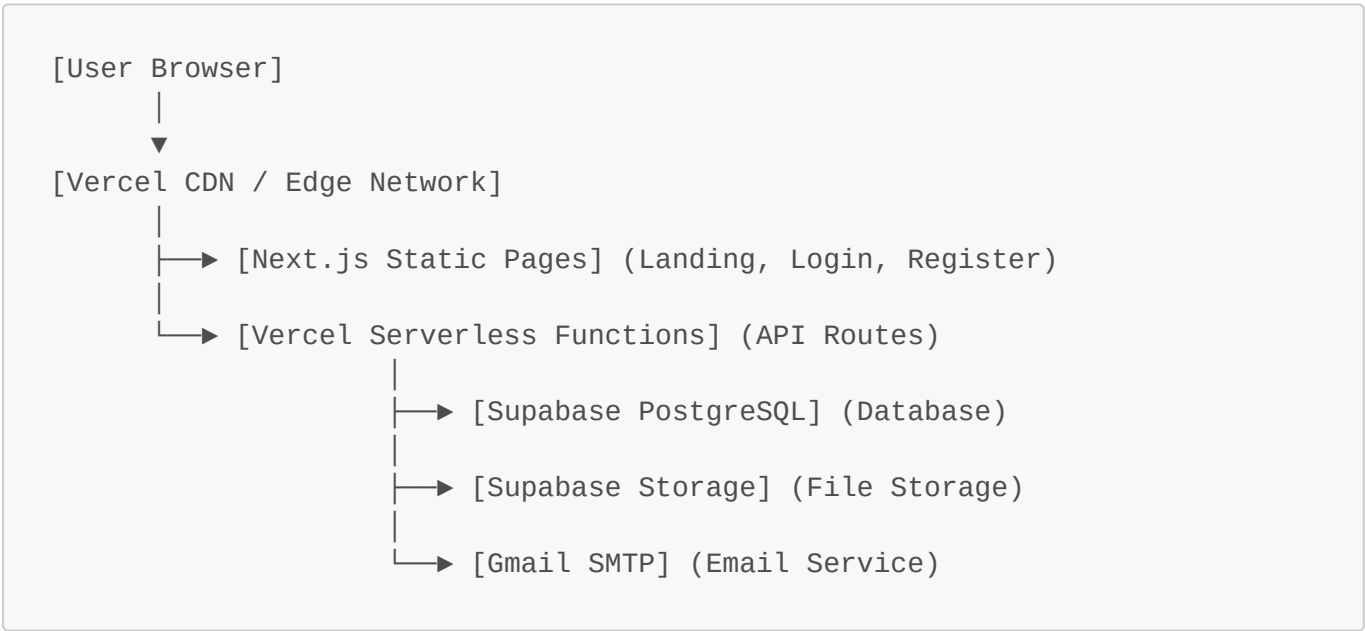


Supabase Storage (File Storage)

Architecture Components

Component	Technology	Responsibility
Frontend	Next.js 16, React 19, Tailwind CSS	UI rendering, user interaction
Authentication	NextAuth.js 4, bcryptjs	Session management, OAuth, 2FA
API Layer	Next.js API Routes (Serverless)	Business logic, data validation
ORM	Prisma 7	Database queries, schema management
Database	PostgreSQL (Supabase)	Persistent data storage
File Storage	Supabase Storage	Medical document storage
Email	Nodemailer (Gmail SMTP)	Verification, OTP, reminders
Deployment	Vercel	Hosting, CI/CD, serverless functions

Deployment Architecture



Task 2: Database Schema Design

Entity Relationship Overview

[Entity Relationship Diagram - MediScript-E]

Database Models

User Model

```
User {
  id          String    (PK, CUID)
  name        String?
  email       String    (Unique)
  password    String
  role        Role      (PATIENT | DOCTOR | ADMIN)
  emailVerified Boolean  (default: false)
  verificationToken String? (Unique)
  verificationExpires DateTime?
  twoFactorEnabled Boolean (default: false)
  twoFactorCode String?
  twoFactorExpires DateTime?
  createdAt   DateTime
  updatedAt   DateTime
}
```

DoctorProfile Model

```
DoctorProfile {
  id          String    (PK, CUID)
  specialization String  (default: "General")
  licenseNo   String    (Unique)
  userId      String    (FK → User.id, Unique)
}
```

PatientProfile Model

```
PatientProfile {
  id          String    (PK, CUID)
  dob         DateTime
  bloodGroup  String    (default: "O+")
  userId      String    (FK → User.id, Unique)
}
```

Appointment Model

```
Appointment {
  id          String    (PK, CUID)
  date        DateTime
  time        String
  reason      String?
  status      String    (PENDING | CONFIRMED | COMPLETED | CANCELLED)
```

```
    doctorId String    (FK → DoctorProfile.id)
    patientId String    (FK → PatientProfile.id)
    createdAt DateTime
    updatedAt DateTime
}
```

Prescription Model

```
Prescription {
  id          String    (PK, CUID)
  diagnosis    String
  medications  String
  doctorId    String    (FK → DoctorProfile.id)
  patientId   String    (FK → PatientProfile.id)
  createdAt   DateTime
}
```

MedicineReminder Model

```
MedicineReminder {
  id          String    (PK, CUID)
  medicineName String
  dosage      String
  frequency   String
  time        String
  startDate   DateTime
  endDate     DateTime
  taken       Boolean   (default: false)
  takenAt     DateTime?
  patientId   String    (FK → PatientProfile.id)
  createdAt   DateTime
  updatedAt   DateTime
}
```

MedicalVault Model

```
MedicalVault {
  id          String    (PK, CUID)
  fileName    String
  fileUrl     String
  patientId   String    (FK → PatientProfile.id)
  createdAt   DateTime
}
```

ContactMessage Model

```
ContactMessage {
  id      String    (PK, CUID)
  name    String
  email   String
  phone   String?
  company String?
  createdAt DateTime
}
```

Task 3: UML Diagrams

3.1 Class Diagram

[Class Diagram - MediScript-E]

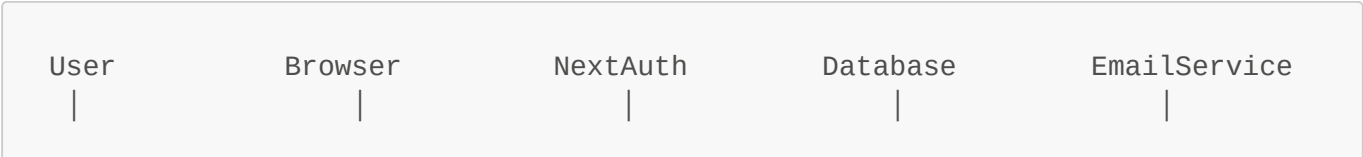
Key Classes and Relationships:

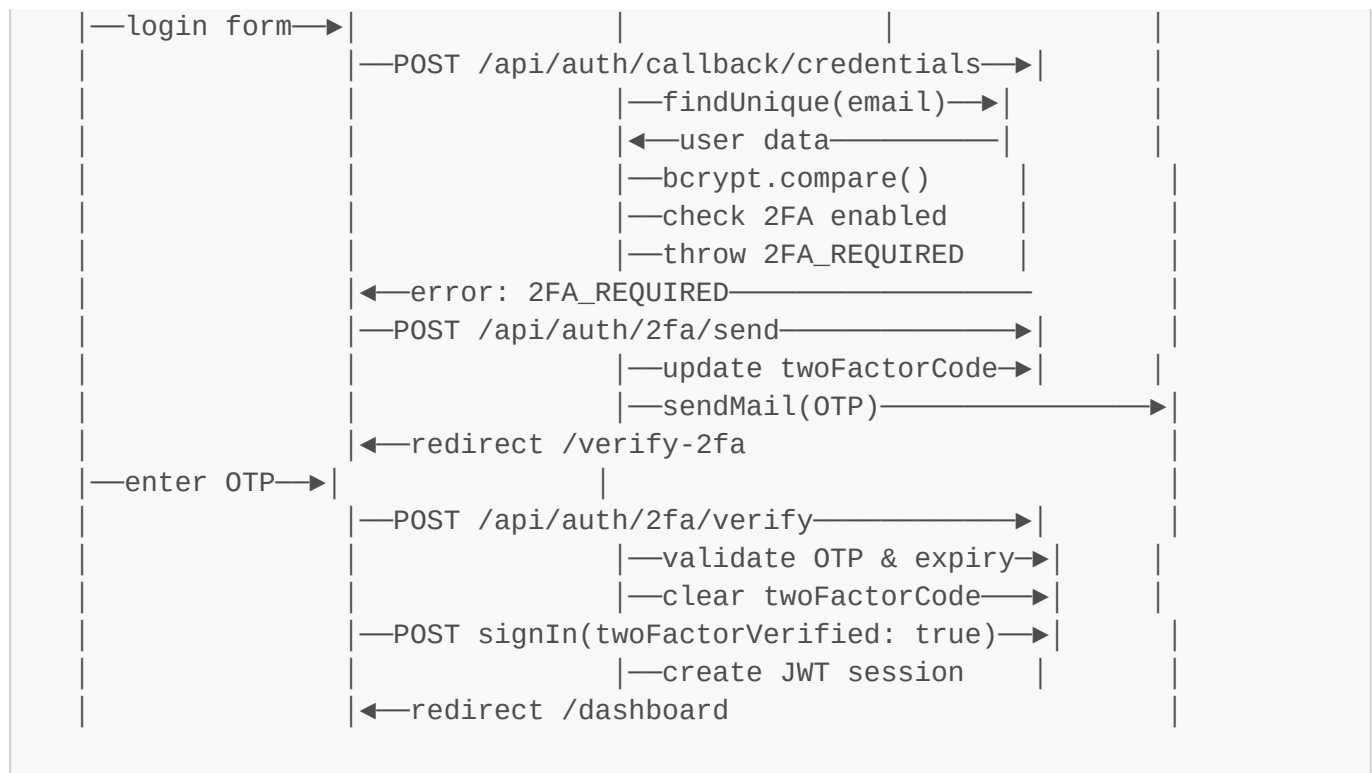
Class	Relationship	Class
User	has one	DoctorProfile
User	has one	PatientProfile
DoctorProfile	has many	Appointment
PatientProfile	has many	Appointment
DoctorProfile	has many	Prescription
PatientProfile	has many	Prescription
PatientProfile	has many	MedicineReminder
PatientProfile	has many	MedicalVault

3.2 Sequence Diagram — User Login with 2FA

[Sequence Diagram - Login with 2FA]

Sequence Description:

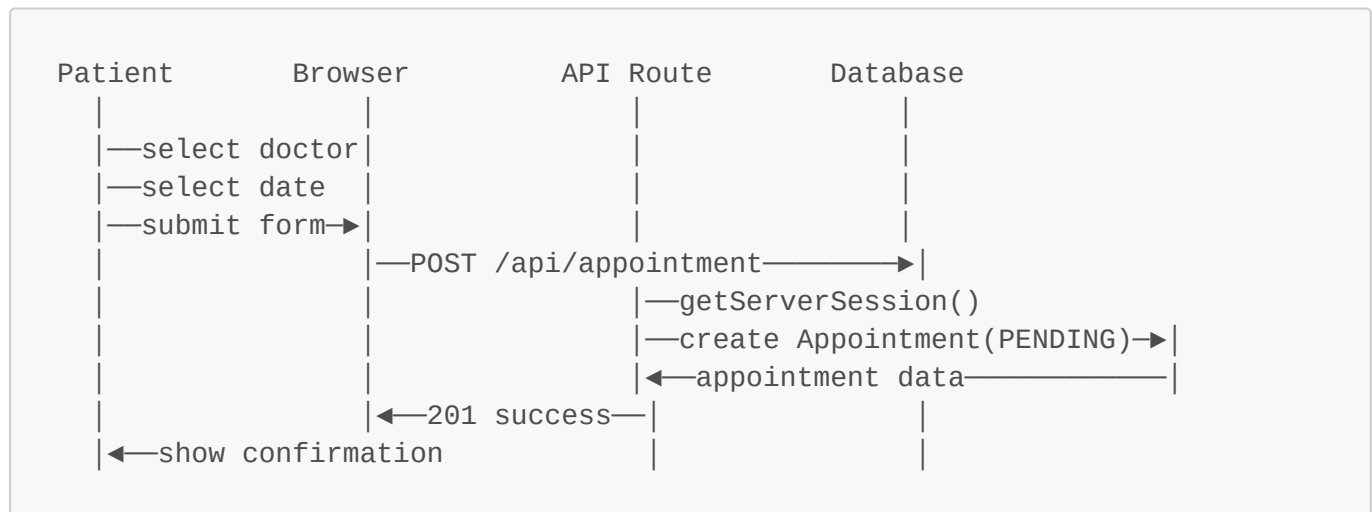




3.3 Sequence Diagram — Book Appointment

[Sequence Diagram - Book Appointment]

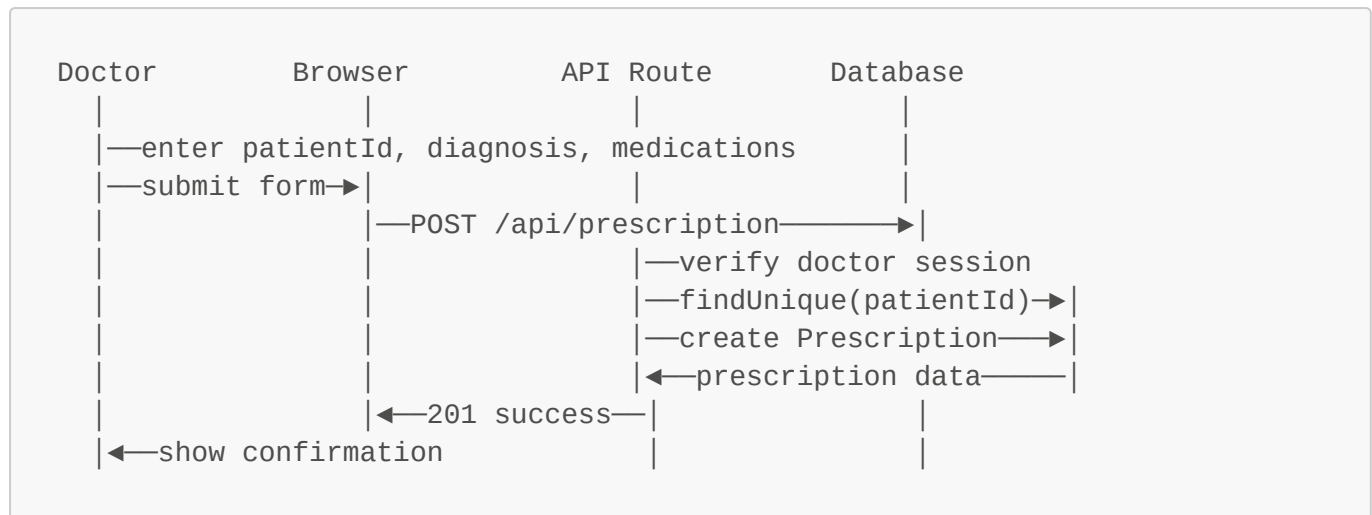
Sequence Description:



3.4 Sequence Diagram — Issue Prescription

[Sequence Diagram - Issue Prescription]

Sequence Description:



Task 4: API Route Design

Authentication APIs

Method	Endpoint	Description	Auth Required
POST	/api/register	Register new user	No
GET/POST	/api/auth/[...nextauth]	NextAuth endpoints	No
POST	/api/verify-email	Verify email token	No
POST	/api/resend-verification	Resend verification email	No
POST	/api/auth/2fa/send	Send OTP email	No
POST	/api/auth/2fa/verify	Verify OTP	No

Patient APIs

Method	Endpoint	Description	Auth Required
GET	/api/appointment	Get user appointments	Yes (Patient)
POST	/api/appointment	Create appointment	Yes (Patient)
PATCH	/api/appointment/[id]	Update appointment status	Yes
DELETE	/api/appointment/[id]	Delete appointment	Yes
GET	/api/doctors	Get all doctors	Yes
POST	/api/prescription	Create prescription	Yes (Doctor)
GET	/api/medicine-reminder	Get reminders	Yes (Patient)
POST	/api/medicine-reminder	Create reminder	Yes (Patient)
PATCH	/api/medicine-reminder/[id]	Mark taken/undo	Yes (Patient)
DELETE	/api/medicine-reminder/[id]	Delete reminder	Yes (Patient)

Method	Endpoint	Description	Auth Required
POST	<code>/api/vault</code>	Upload medical document	Yes (Patient)
DELETE	<code>/api/vault/[id]</code>	Delete document	Yes (Patient)

Settings APIs

Method	Endpoint	Description	Auth Required
PATCH	<code>/api/settings/profile</code>	Update profile name	Yes
PATCH	<code>/api/settings/password</code>	Change password	Yes
PATCH	<code>/api/settings/2fa</code>	Toggle 2FA	Yes

Admin APIs

Method	Endpoint	Description	Auth Required
GET	<code>/api/admin/stats</code>	Get dashboard statistics	Yes (Admin)
GET	<code>/api/admin/users</code>	Get all users	Yes (Admin)
DELETE	<code>/api/admin/users/[id]</code>	Delete user	Yes (Admin)
GET	<code>/api/admin/appointments</code>	Get all appointments	Yes (Admin)
GET	<code>/api/admin/contacts</code>	Get contact messages	Yes (Admin)

Key Findings / Learning Outcomes

- Designed a clean **3-tier architecture** separating presentation, business logic, and data layers
- Applied **Next.js App Router** conventions for both frontend and serverless API design
- Modeled **8 database entities** with proper relationships using Prisma schema
- Created UML sequence diagrams for critical workflows: Login with 2FA, Book Appointment, Issue Prescription
- Understood how architectural decisions (serverless, connection pooling, SSL) directly impact production reliability
- Recognized that low coupling between modules (auth, appointments, prescriptions, vault) enables independent development and testing